

Supplementary IV

(Aggregate Signature Schemes and Cryptanalysis Implementation)

This project implements Xu’s scheme and two cryptanalysis attacks. A total of 3 C code files are developed based on the PBC cryptography library. All schemes generate 40 user key pairs and 40 individual signatures.

The attack algorithms rely on the Wagner algorithm (a probabilistic birthday search algorithm). To ensure deterministic attack results, we modify the hash generation logic and replace probabilistic Wagner searching with fixed hash summation constraints.

1 Source Code Files (Total: 3)

- `Xu.c`: Implementation of the Xu aggregate signature scheme
- `XuA1.c`: Type A1 attack on the Xu scheme
- `XuA2.c`: Type A2 attack on the Xu scheme

2 Function Descriptions

2.1 Original Schemes (`Xu.c`)

Each program performs the following operations:

1. Generate 40 pairs of user keys
2. Generate 40 individual signatures
3. Verify 40 partial private keys
4. Verify 40 individual signatures separately
5. Aggregate 40 signatures and verify the aggregate signature

2.2 Attack on Xu (`XuA1.c`, `XuA2.c`)

- TA only generates key pairs for users ($V_1 \sim V_{39}$)
- **A1 Attack**: Adversary forges public key, identity, and pseudonym for (V_0)
- **A2 Attack**: Adversary cannot modify public key, identity, or pseudonym for (V_0)

3 Key Modification: Wagner Algorithm Replacement

Xu’s scheme is attacked using the Wagner algorithm, which is **probabilistic** and cannot guarantee success in every run.

To achieve deterministic and stable attack results, we modify the hash generation logic:

1. In XuA1.c and XuA2.c: Force the sum of hash values to be a **fixed independent hash value**
2. The original probabilistic Wagner algorithm implementations are preserved in TestH.c

4 Important Compatibility Notes

4.1 Differences in Hash Bit Length Design

- Tool codes: TestH.c adopt 64-bit hash length, which is designed for efficient verification of the Wagner algorithm.
- Attack codes: XuA1.c / XuA2.c adopt 256-bit hash length, aiming to maintain consistency with the standard parameters in the manuscript.

4.2 Incompatibility Conclusion

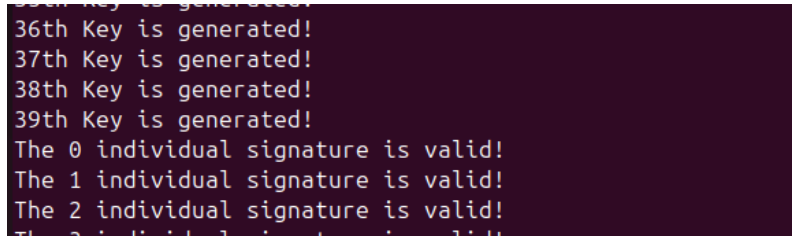
Due to the obvious difference in hash bit lengths, TestH.c cannot be directly compatible or mixed with XuA1.c and XuA2.c.

5 Screenshot Requirements

5.1 Original Schemes

Include three verification results:

1. Verification of 40 partial private keys
2. Verification of 40 individual signatures
3. Verification of the final aggregate signature



```

36th Key is generated!
37th Key is generated!
38th Key is generated!
39th Key is generated!
The 0 individual signature is valid!
The 1 individual signature is valid!
The 2 individual signature is valid!
The 3 individual signature is valid!

```

5.2 Attack Programs (All 2 Files)

The TA generates 39 partial private keys for $\mathcal{A}_1$ in Xu's scheme, but $\mathcal{A}_1$ still can forge an aggregate signature for 40 vehicles.

```
The 35 individual signature is valid!  
The 36 individual signature is valid!  
The 37 individual signature is valid!  
The 38 individual signature is valid!  
The 39 individual signature is valid!  
The e_left is [38782387533914297375094075052657079705357  
818127, 137412958985023755697661452341668874904336623387  
The e_right is [3878238753391429737509407505265707970535  
2818127, 13741295898502375569766145234166887490433662338  
]  
The aggregate signature is valid!
```

```
35th Key is generated!  
36th Key is generated!  
37th Key is generated!  
38th Key is generated!  
39th Key is generated!  
The e_left is [188587802182270781301039928539498074  
075024, 2382551023308027908098209071565419176507762  
The e_right is [18858780218227078130103992853949807  
7075024, 238255102330802790809820907156541917650776  
]  
The aggregate signature is valid!
```

6 Compilation Notes

All programs depend on the PBC library and GMP library.

6.1 Compilation Command

```
gcc -o Xu Xu.c -lpbc -lgmp -lcrypto  
gcc -o XuA1 XuA1.c -lpbc -lgmp -lcrypto  
gcc -o XuA2 XuA2.c -lpbc -lgmp -lcrypto
```

6.2 Execution Command

```
./Xu  
./XuA1  
./XuA2
```